
Python 金融数据分析

第 4 章 *pandas* 数据分析

主讲人： 王伟冠, 吕文俊
院 系： 经济学院金融系

上海大学

2026 年 9 月 24 日



1. 初识 pandas
2. Series: 带标签的一列数据
3. DataFrame: 金融数据的“主战场”
4. 数据选取与筛选
5. 排序与分组
6. 数据清洗与输入输出
7. 合并与长宽表转换
8. 分组统计与综合应用
9. 金融时间序列
10. 缺失值处理
11. 读写数据
12. 综合案例与小结



1. 初识 pandas



- ① 表与索引：选择、赋值、对齐，先说明一行代表什么。
- ② 清洗：类型、重复、缺失、异常，保留处理依据。
- ③ 合并与变形：连接关系、行数变化、长表与宽表。
- ④ 分组与应用：聚合、组内计算、订单日报与金融迁移。

课堂节奏

每单元可安排一周，也可将讲解、上机和讲评分开。完成本单元练习后再推进，不要求一周讲完整章。

- 单元一：04_pandas 第 1-5 节。
- 单元二：04_2_Cleaning, 整理成绩登记。
- 单元三：04_3_Wrangling, 合并订单与商品。
- 单元四：04_4_GroupBy, 完成订单日报;
再回到 04_pandas 第 6-9 节, 完成金融案例。

运行方式

三个补充 Notebook 均可单独从头运行。返回主 Notebook 时从头执行, 恢复本文件的变量。

- 理解 Series 与 DataFrame 两种核心结构
- 掌握数据的选取：按列、按标签 loc、按位置 iloc、条件筛选
- 掌握排序与分组聚合 (groupby)
- 掌握金融时间序列的基本操作：日期索引、pct_change、重采样
- 会处理缺失值，会用 read_csv 读入真实数据
- 掌握清洗、连接、长宽表转换与组内统计，能够对一张股价表完成“收益率—风险—相关性”的完整分析

前置知识

第 2 章的字典/列表、第 3 章的 NumPy——pandas 就建在 NumPy 之上。

- 第 3 章我们算出了收益率数组，但马上会遇到新问题：
 - 每个数字对应哪一天？——NumPy 数组没有标签；
 - 行情数据里有日期、代码、名称——普通数组通常使用统一的数据类型；
 - 数据有缺失（停牌）怎么办？
 - 从 Excel/CSV 读进来的表格如何直接分析？
- **pandas** 就是为此而生：给数组加上行列标签，成为“数据表”。

一个直观比喻

NumPy 数组像一张写满数字的草稿纸；
pandas DataFrame 像一张带表头和日期列的 Excel 表，而且能用代码操作。

```
1 import numpy as np
2 import pandas as pd      # 约定俗成，照抄
```

结构	可以理解为	金融中的例子
Series	一列带索引的数据	某股票的收盘价序列
DataFrame	一张带行列标签的表	多股票 × 多日期的价格表

- DataFrame 本质上就是“共享同一个索引的若干 Series”。
- 本章几乎所有内容都围绕这张“表”的查看、选取、变形与统计。

2. Series: 带标签的一系列数据



```
1 s = pd.Series([1500.35, 128.60, 55.20],  
2               index=["600519", "000858", "601318"])  
3 s
```

```
1 600519    1500.35  
2 000858     128.60  
3 601318      55.20  
4 dtype: float64
```

- 左边是**索引** (index, 相当于标签), 右边是值。
- 不指定索引时, 默认为 0, 1, 2, ... (像列表)。
- 索引可以是股票代码、日期、任何你需要的标签。

```
1 s["600519"]          # 1500.35 按标签取值
2 s[["600519", "601318"]] # 取多个
3
4 s * 1.05              # 所有值涨5% (向量化, 继承自NumPy)
5 s[s > 100]            # 筛选出大于100的
```

- Series 支持 NumPy 式向量化运算与布尔筛选——旧知识直接复用。
- 取值时用的是**标签**而不是位置：s["600519"]。

关键概念：对齐

两个 Series 运算时按**索引对齐**，而不是按位置：

" 茅台的价格" 只和" 茅台的权重" 相乘，顺序打乱也不会错。

3. DataFrame: 金融数据的"主战场"



```
1 data = {"茅台": [1500.0, 1520.0, 1515.0],  
2         "五粮液": [128.0, 129.5, 127.0],  
3         "平安": [55.0, 54.5, 56.0]}  
4 df = pd.DataFrame(data,  
5                     index=["2026-07-01", "2026-07-02", "2026-07-03"])  
6 df
```

- 字典的键变成列名，列表里的值成为该列数据。
- 约定俗成的金融数据方向：行 = 日期，列 = 资产。
- Notebook 中直接写变量名 df，会以漂亮的表格显示。

```
1 df.shape      # (3, 3)      行数、列数
2 df.columns    # 列名: Index(['茅台', '五粮液', '平安'])
3 df.index      # 行索引 (这里是日期)
4 df.head(2)    # 前2行 (head()默认5行)
5 df.tail(1)    # 最后1行
6 df.describe() # 每列的统计摘要: 均值、标准差、分位数等
```

- 拿到任何新数据, 先把这些“体检动作”做一遍。
- describe() 相当于免费的初步报告, 务必常用。

这张表每行代表一次学生成绩记录，学号是标识而非数量。金融表也可以每行代表“股票—日期”，不必总是每列一只股票。

```
1 students = pd.DataFrame({
2     "学号": ["001", "002", "003", "004"],
3     "姓名": ["小林", "小周", "小陈", "小许"],
4     "班级": ["A", "A", "B", "B"],
5     "成绩": [82, 95, 68, 88]})
6 students.info()
7 students.head()
```

单层方括号取一列得到 Series；列名列表得到 DataFrame。先观察 type 和 shape，再决定后面如何操作。

```
1 one_column = students["成绩"]
2 one_table = students[["成绩"]]
3 print(type(one_column), one_column.shape)
4 print(type(one_table), one_table.shape)
5 print(students.describe())
```

4. 数据选取与筛选



```
1 df["茅台"]           # 取一列 -> Series
2 df[["茅台", "平安"]] # 取多列 -> DataFrame
3 df["2026-07-01"]     # 报错! 方括号默认是"按列"
```

- `df[...]` 里放列名：取列。
- 想按行取，用切片（按位置）或下一页的 `loc/iloc`。

```
1 df[0:2]              # 前两行（位置切片，含头不含尾）
```

最常见的困惑

为什么 `df["茅台"]` 可以，`df["2026-07-01"]` 却报错？
因为 `[]` 默认针对**列**。规则记牢，少查一半报错。

```
1 df.loc["2026-07-02"]           # 该行全部列
2 df.loc["2026-07-02", "茅台"]   # 指定行、指定列 -> 1520.0
3 df.loc[:, "茅台"]              # 所有行、茅台列
4
5 df.iloc[0]                     # 第0行（按位置）
6 df.iloc[0:2, 1]                # 前2行、第1列
7 df.iloc[-1]                    # 最后一行
```

- loc: 用**标签** (日期、列名); 写法 [行, 列]。
- iloc: 用**位置** (0, 1, 2...); 与列表索引规则相同。
- 先想清楚“我要按名字找还是按位置找”, 再选工具。

```
1 df[df["茅台"] > 1510]          # 茅台价格高于1510的日子
2 df[(df["茅台"] > 1500) & (df["平安"] > 55)]
```

- 与 NumPy 布尔索引完全一致：先算出布尔列，再用来筛行。
- 多条件组合用 &、|，每个条件加括号。
- 金融例子：涨幅超过 5% 的日子、PE 低于 10 且股息率高于 4% 的股票。

课堂练习

用一行代码选出“五粮液价格低于 128”的所有交易日。

学号设为索引后, loc 用学号找人, iloc 按当前位置找人。在此唯一且有序的索引上, loc 切片包括末端标签, iloc 不包括末端位置。

```
1 by_id = students.set_index("学号")
2 print(by_id.loc["001":"003", ["姓名", "成绩"]])
3 print(by_id.iloc[0:2, [0, 2]])
4 print(by_id.reset_index().columns.tolist())
```

用 loc 一次指定行与列进行赋值；不要在 df[条件][列名] 的临时结果上修改原表。保留原始成绩，另建调整列。

```
1 students["调整成绩"] = students["成绩"]
2 mask = students["班级"].eq("B")
3 students.loc[mask, "调整成绩"] = students.loc[mask, "成绩"] + 2
4 selected = students.loc[students["成绩"].ge(80),
5                          ["姓名", "班级", "成绩"]].copy()
6 print(selected)
```

筛选 A 班且成绩至少为 90 的学生，仅展示姓名与成绩。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

Series 运算按标签匹配。缺少对应项会得到缺失值；fill_value=0 只适合“未出现确实表示没有”的业务含义。

```
1 morning = pd.Series([3, 5], index=["茶", "咖啡"])
2 afternoon = pd.Series([2, 4], index=["咖啡", "果汁"])
3 print(morning + afternoon)
4 print(morning.add(afternoon, fill_value=0))
```

5. 排序与分组



```
1 df.sort_values("茅台", ascending=False) # 按茅台价格降序
2 df.sort_index() # 按行索引(日期)排序
```

- `sort_values(列名)`: 按某列的值给**行**排序。
- `sort_index()`: 按行索引排序——日期数据常用。
- 排序返回**新表**, 原表不变 (除非加 `inplace=True`)。

习惯

时间序列数据到手先 `sort_index()`, 保证日期升序, 后续计算才安全。

```
1 stocks = pd.DataFrame({
2     "代码": ["600519", "000858", "601318", "600036"],
3     "行业": ["白酒", "白酒", "保险", "银行"],
4     "市盈率": [28.5, 18.2, 8.1, 6.3]})
5
6 stocks.groupby("行业")["市盈率"].mean()
```

```
1 行业
2 保险      8.10
3 银行      6.30
4 白酒     23.35
```

- 三步走：分组 (groupby) → 选列 → 聚合 (mean/sum/max...)。
- 回答“每个行业的平均市盈率”这类问题，一行搞定。

6. 数据清洗与输入输出



教学合成数据，每行一次成绩登记。故意保留首尾空格、重复记录、缺考和不合理成绩；学号按字符串读入以保留前导零。

```
1 from io import StringIO
2 import pandas as pd
3 raw_csv = """学号,姓名,班级,成绩,日期
4 001, 小林 ,A,82,2026-09-01
5 002,小周,a,95,2026-09-01
6 003,小陈,B,缺考,2026-09-02
7 004,小许,B,108,2026-09-02
8 001, 小林 ,A,82,2026-09-01
9 005,小吴, B ,76,日期待核"""
10 raw = pd.read_csv(StringIO(raw_csv), dtype="string")
11 raw
```

info 看类型和非缺失数，duplicated 找整行重复。这里“缺考”是普通文本，尚不会被 isna 自动识别。

```
1 raw.info()
2 print(raw.isna().sum())
3 print("整行重复数: ", raw.duplicated().sum())
4 raw.loc[raw.duplicated(keep=False)]
```

copy 保留原始登记。统一班级格式后再分组，否则 A 和 a 会被算成两个班。

```
1 clean = raw.copy()
2 clean["姓名"] = clean["姓名"].str.strip()
3 clean["班级"] = clean["班级"].str.strip().str.upper()
4 print(clean["班级"].value_counts())
5 print(clean["姓名"].str.contains("小", na=False))
```

先把明确的缺考编码改成缺失，再转数值。coerce 将无法解析的值变成缺失；保留原列才能区分缺考与格式错误。

```
1 score_text = clean["成绩"].replace("缺考", pd.NA)
2 clean["分数"] = pd.to_numeric(score_text, errors="coerce")
3 clean["登记日"] = pd.to_datetime(clean["日期"], errors="coerce")
4 parse_failed = score_text.notna() & clean["分数"].isna()
5 print(clean.loc[parse_failed | clean["登记日"].isna()])
6 print(clean.dtypes)
```

本例每名学生只有一次考试，学号应唯一。已确认整行相同的重复登记可以删除；若同一学号分数不同，应先核实，不能任意保留。

```
1 print(clean.loc[clean.duplicated("学号", keep=False)])  
2 clean = clean.drop_duplicates().reset_index(drop=True)  
3 print("清理后记录数: ", len(clean))  
4 print("学号唯一: ", clean["学号"].is_unique)
```

考试满分 100，因此 108 是待核异常，不能直接截成 100。缺考也不等于 0 分。设置标记后，在统计列中排除异常。

```
1 clean["成绩异常"] = (clean["分数"].notna() &  
2                      ~clean["分数"].between(0, 100))  
3 clean["有效成绩"] = clean["分数"].mask(clean["成绩异常"])  
4 print(clean[["学号", "成绩", "成绩异常", "有效成绩"]])  
5 print("有效成绩均值: ", clean["有效成绩"].mean())
```

计算有效成绩数量、缺考数量、异常成绩数量。为什么不能把三者混成一类？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

比较同一列的有效值平均与填零平均。dropna 的 subset 明确本次分析依赖哪些列；不能因为日期缺失就删除不依赖日期的成绩。

```
1 print(clean["有效成绩"].mean())
2 print(clean["有效成绩"].fillna(0).mean())
3 usable_scores = clean.dropna(subset=["有效成绩"])
4 print(usable_scores[["学号", "有效成绩"]])
```

cut 按固定阈值分段，right=False 表示左闭右开。100 包含在 [90,101) 内；没有有效成绩的行仍保留缺失等级。

```
1 clean["等级"] = pd.cut(clean["有效成绩"],
2     bins=[0, 60, 80, 90, 101], right=False,
3     labels=["待提高", "合格", "良好", "优秀"])
4 clean["班级名称"] = clean["班级"].map({"A": "一班", "B": "二班"})
5 clean[["学号", "有效成绩", "等级", "班级名称"]]
```

先在内存中演示 CSV 往返。读回时显式指定学号类型；分类类型和日期类型需要重建。
真实文件路径可替换 StringIO。

```
1 csv_text = clean.to_csv(index=False)
2 restored = pd.read_csv(StringIO(csv_text), dtype={"学号": "string"})
3 print(restored["学号"].tolist())
4 print(len(row), len(clean), len(restored))
5 assert len(row) == 6 and len(clean) == 5
6 assert restored["学号"].iloc[0] == "001"
```

交付三项结果：保留几行、排除哪些成绩、哪些日期待核。下一单元学习把这类记录与另一张信息表合并。

```
1 audit = pd.Series({
2     "输入行数": len(raw), "保留行数": len(clean),
3     "有效成绩数": clean["有效成绩"].count(),
4     "异常成绩数": clean["成绩异常"].sum(),
5     "待核日期数": clean["登记日"].isna().sum()})
6 print(audit)
```

找出有效成绩低于 80 的学生，保留学号、姓名、班级、有效成绩。解释为什么没有把缺考者选进来。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

7. 合并与长宽表转换



合成订单表每行一笔订单，商品表每行一种商品。同一商品可以出现在多笔订单中，因此合并关系是多对一。

```
1 import pandas as pd
2 orders = pd.DataFrame({
3     "订单": [1, 2, 3, 4], "商品": ["T", "C", "T", "X"],
4     "数量": [2, 1, 3, 1]})
5 products = pd.DataFrame({
6     "商品": ["T", "C", "J"],
7     "名称": ["茶", "咖啡", "果汁"], "单价": [10, 20, 15]})
8 print(orders)
9 print(products)
```

on 明确匹配键；left 保留左表订单。indicator 标记未匹配行，validate 检查商品表的键唯一，避免合并后订单被复制。

```
1 joined = orders.merge(products, on="商品", how="left",
2     validate="many_to_one", indicator=True)
3 print(joined)
4 print(joined["_merge"].value_counts())
5 assert len(joined) == len(orders)
```

inner 只保留双方存在的商品匹配；outer 还会显示没有订单的商品。未匹配不代表金额为零，应单独列出。

```
1 inner = orders.merge(products, on="商品", how="inner")
2 outer = orders.merge(products, on="商品", how="outer",
3                       indicator=True)
4 print(len(inner), len(outer)) # 3, 5
5 print(outer[["订单", "商品", "_merge"]])
```

哪笔订单在 inner 中消失？哪个商品只在右表中？如果要核对全部订单，该选哪种连接？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

在商品表中故意重复 T，观察两笔 T 订单各匹配两次。解决方法是核实键与版本，而不是合并后随意去重。

```
1 bad_products = pd.concat([products, products.iloc[[0]]],  
2                           ignore_index=True)  
3 expanded = orders.merge(bad_products, on="商品", how="left")  
4 print(len(orders), len(expanded)) # 4, 6  
5 print(bad_products.duplicated("商品").sum())  
6 # 加 validate="many_to_one" 将直接报错，课堂可自行尝试。
```

concat 不按商品键寻找匹配。ignore_index 重建行号, 不检查业务编号是否重复; 拼接后仍要检查订单号。

```
1 next_orders = pd.DataFrame({
2     "订单": [5, 6], "商品": ["C", "T"], "数量": [2, 1]})
3 all_orders = pd.concat([orders, next_orders], ignore_index=True)
4 print(all_orders)
5 print(all_orders["订单"].is_unique)
```

`axis=1` 横向拼接时按索引标签对齐，不会因为行数相同就按位置贴在一起。先检查索引含义。

```
1 left = pd.Series([80, 90], index=["001", "002"], name="数学")
2 right = pd.Series([70, 85], index=["002", "001"], name="英语")
3 print(pd.concat([left, right], axis=1))
```

长表每行是一名学生的一门课成绩；宽表每行一名学生、每列一门课。pivot 要求“学号—课程”组合唯一。

```
1 long_scores = pd.DataFrame({
2     "学号": ["001", "001", "002", "002"],
3     "课程": ["数学", "英语", "数学", "英语"],
4     "成绩": [80, 90, 70, 85]})
5 wide = long_scores.pivot(index="学号", columns="课程", values="成绩")
6 print(wide)
```

`id_vars` 指定不被展开的标识列。`melt` 将列名变成课程字段，将原来的单元格变成成绩字段。

```
1 back = wide.reset_index().melt(id_vars="学号",  
2     var_name="课程", value_name="成绩")  
3 print(back.sort_values(["学号", "课程"]))  
4 assert len(back) == len(long_scores)
```

加入一次补考后，同一学生同一课程出现两次。pivot_table 可以聚合；取最大值代表“保留最高成绩”，这是业务规则。

```
1  retake = pd.DataFrame({"学号": ["002"],
2                          "课程": ["数学"], "成绩": [88]})
3  attempts = pd.concat([long_scores, retake], ignore_index=True)
4  best = attempts.pivot_table(index="学号", columns="课程",
5                               values="成绩", aggfunc="max")
6  print(best)
```

将 max 改为 mean, 002 的数学成绩变成多少？两种答案分别回答什么问题？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

MultiIndex 可以表示“学号—课程”这类组合标签。先学 `set_index` 和 `reset_index`，再选学 `stack`、`unstack`。

```
1 indexed = long_scores.set_index(["学号", "课程"]).sort_index()
2 print(indexed.loc[("001", "数学"), "成绩"])
3 print(indexed.unstack("课程"))
4 print(indexed.reset_index())
```

价格未知的 X 订单不能当作零金额。先列出未匹配订单，再汇总可计算的部分，并说明覆盖范围。

```
1 matched = joined.loc[joined["_merge"] == "both"].copy()
2 matched["金额"] = matched["数量"] * matched["单价"]
3 print(matched[["订单", "名称", "金额"]])
4 print("已知金额合计: ", matched["金额"].sum()) # 70
5 print("未匹配订单数: ", (joined["_merge"] == "left_only").sum())
```

将商品 X 补录为“水”、单价 5，重新合并全部原订单。总金额应是多少？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

8. 分组统计与综合应用



六笔合成订单用于手工核对。金额为数量乘单价；本例价格均已匹配，日期是实际有订单的日期。

```
1 import pandas as pd
2 sales = pd.DataFrame({
3     "订单": [1, 2, 3, 4, 5, 6],
4     "日期": ["2026-09-01"] * 3 + ["2026-09-02"] * 3,
5     "门店": ["东", "东", "西", "东", "西", "西"],
6     "商品": ["茶", "咖啡", "茶", "茶", "咖啡", "茶"],
7     "数量": [2, 1, 1, 3, 2, 2], "单价": [10, 20, 10, 10, 20, 10]})
8 sales["日期"] = pd.to_datetime(sales["日期"])
9 sales["金额"] = sales["数量"] * sales["单价"]
10 sales
```

先按门店拆成小表，在各小表内求和，再组成汇总表。as_index=False 使门店保留为普通列，便于继续合并。

```
1 store_total = sales.groupby("门店", as_index=False)["金额"].sum()
2 print(store_total)
3 print(sales.groupby(["门店", "商品"])["金额"].sum())
```

总金额、订单数和平均每单金额是不同指标。命名聚合直接给结果列起名；金额均值的分母是订单行数。

```
1 summary = sales.groupby("门店", as_index=False).agg(  
2     总金额=("金额", "sum"),  
3     订单数=("订单", "size"),  
4     总杯数=("数量", "sum"),  
5     平均每单金额=("金额", "mean"))  
6 print(summary)
```

size 统计组内行数, count 统计某列非缺失值数。先明确要数订单、有效金额还是不同顾客, 再选择统计方法。

```
1 with_missing = sales.copy()
2 with_missing.loc[0, "金额"] = float("nan")
3 counts = with_missing.groupby("门店").agg(
4     行数=("金额", "size"), 有效金额数=("金额", "count"))
5 print(counts)
```

东店的行数与有效金额数各是多少？金额缺失能否当作免费订单？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

平均每单金额不等于平均每杯价格。后者用总金额除以总杯数，相当于按购买数量给单价加权。

```
1 summary["平均每杯价格"] = summary["总金额"] / summary["总杯数"]
2 print(summary[["门店", "平均每单金额", "平均每杯价格"]])
3 print("全体每杯均价：", sales["金额"].sum() / sales["数量"].sum())
```

agg 每组输出一行, transform 返回与原表等长且对齐的结果。用每笔金额除以所属门店总金额得到组内占比。

```
1 sales["门店总金额"] = sales.groupby("门店")["金额"].transform("sum")
2 sales["门店内占比"] = sales["金额"] / sales["门店总金额"]
3 print(sales[["订单", "门店", "金额", "门店内占比"]])
4 print(sales.groupby("门店")["门店内占比"].sum())
```

先按金额降序，再按订单号升序决定并列次序，组内取前两行。结果是两笔订单，而不一定是两个不同金额档位。

```
1 ordered = sales.sort_values(["门店", "金额", "订单"],
2                             ascending=[True, False, True])
3 top2 = ordered.groupby("门店").head(2)
4 print(top2[["订单", "门店", "金额"]])
5 sales["组内名次"] = sales.groupby("门店")["金额"].rank(
6     ascending=False, method="min")
```

crosstab 默认数记录。pivot_table 显式指定金额与 sum 才是金额汇总。本例已知交易明细完整，因此没有订单的组合可填零。

```
1 print(pd.crosstab(sales["门店"], sales["商品"]))
2 amount_table = sales.pivot_table(index="门店", columns="商品",
3     values="金额", aggfunc="sum", fill_value=0)
4 print(amount_table)
```

先汇总到“日”，再滚动才是相邻观测日的窗口。rolling(2) 是两行，不自动补齐日历中没有记录的日期。

```
1 daily = sales.groupby("日期")["金额"].sum().sort_index()
2 print(daily)
3 print(daily.rolling(2, min_periods=2).mean())
4 print(sales.set_index("日期")["金额"].resample("D").sum(min_count=1))
```

合成长表每行一只股票一天的收盘价。先按股票和日期排序，在每只股票内部错位，避免把另一只股票的价格当作昨日价。

```
1 quotes = pd.DataFrame({
2     "股票": ["A", "B", "A", "B", "A", "B"],
3     "日期": pd.to_datetime(["2026-09-01"] * 2 +
4                             ["2026-09-02"] * 2 + ["2026-09-03"] * 2),
5     "收盘价": [100, 50, 102, 49, 101, 50]})
6 quotes = quotes.sort_values(["股票", "日期"])
7 quotes["昨日价"] = quotes.groupby("股票")["收盘价"].shift(1)
8 quotes["收益率"] = quotes["收盘价"] / quotes["昨日价"] - 1
9 print(quotes)
```

pivot 后每列一只股票；pct_change(fill_method=None) 不自动填补缺失价格。比较两种写法的同一观测值。

```
1 price_matrix = quotes.pivot(index="日期", columns="股票", values="收盘价")
2 return_matrix = price_matrix.pct_change(fill_method=None)
3 print(return_matrix)
4 expected = quotes.loc[(quotes["股票"] == "A") &
5                       (quotes["日期"] == "2026-09-02"), "收益率"].iloc[0]
6 assert abs(return_matrix.loc["2026-09-02", "A"] - expected) < 1e-12
```

先用订单手算，再检查汇总表。全表金额应为 140 元、总杯数为 11；两门店各 70 元，但每杯均价不同。

```
1 assert sales["金额"].sum() == 140
2 assert sales["数量"].sum() == 11
3 assert summary["总金额"].sum() == sales["金额"].sum()
4 print(summary)
```

按“日期—门店”统计金额与杯数，并计算每杯均价。哪一天全体销售金额更高？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

下面重新开始一套独立的合成数据。每行原本应是一笔订单，但有重复登记、数量缺失和未知商品；目标是输出可核对的已知金额日报。

```
1 raw_orders = pd.DataFrame({
2     "订单": ["01", "02", "03", "04", "05", "06", "01"],
3     "日期": ["2026-09-01"] * 3 + ["2026-09-02"] * 3 + ["2026-09-01"],
4     "门店": ["东", "东", "西", "东", "西", "西", "东"],
5     "商品": ["t", "C", "T", "T", "C", "X", "t"],
6     "数量": ["2", "1", "缺失", "3", "2", "1", "2"]})
7 catalog = pd.DataFrame({"商品": ["T", "C"], "单价": [10, 20]})
8 raw_orders
```

保留 `raw_orders`。先统一商品代码，再删除完全相同的登记，转换日期和数量；原始数量列保留，便于回查缺失原因。

```
1 tidy_orders = raw_orders.copy()
2 tidy_orders["商品"] = tidy_orders["商品"].str.strip().str.upper()
3 tidy_orders = tidy_orders.drop_duplicates().reset_index(drop=True)
4 tidy_orders["日期"] = pd.to_datetime(tidy_orders["日期"])
5 tidy_orders["有效数量"] = pd.to_numeric(tidy_orders["数量"], errors="coerce")
6 assert tidy_orders["订单"].is_unique
7 print(tidy_orders)
```

左连接保留每笔订单。已匹配商品且数量为正才进入本例金额统计；数量缺失和价格未知都保留在待核清单中。

```
1 enriched = tidy_orders.merge(catalog, on="商品", how="left",  
2     validate="many_to_one", indicator=True)  
3 eligible = enriched["_merge"].eq("both") & enriched["有效数量"].gt(0)  
4 pending = enriched.loc[~eligible].copy()  
5 known = enriched.loc[eligible].copy()  
6 known["金额"] = known["有效数量"] * known["单价"]  
7 print(pending[["订单", "数量", "商品", "_merge"]])
```

已知金额日报仅覆盖四笔订单，不能称为全部销售额。汇总前后金额应一致，同时报告两笔待核订单。

```
1 known_report = known.groupby(["日期", "门店"], as_index=False).agg(  
2     已知金额=("金额", "sum"), 可计算订单数=("订单", "size"))  
3 print(known_report)  
4 print("待核订单数: ", len(pending))  
5 assert len(raw_orders) == 7 and len(tidy_orders) == 6  
6 assert len(known) == 4 and len(pending) == 2  
7 assert known_report["已知金额"].sum() == 110
```

日报透视成“日期 × 门店”方便比较。西店首日有数量待核订单，不能把该格写成零销售；即使已有金额的格子也可能尚未完整。

```
1 known_wide = known_report.pivot(index="日期", columns="门店", values="已知金额")
2 pending_wide = pending.groupby(["日期", "门店"]).size().unstack("门店")
3 print("已知金额（不是完整销售额）：")
4 print(known_wide)
5 print("待核订单数：")
6 print(pending_wide)
```

核实订单 03 数量为 1、商品 X 是 5 元的水。重新清洗合并并汇总，完整金额应为 125 元。不要在最终报表中直接改金额。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

9. 金融时间序列



```
1 df.index = pd.to_datetime(df.index)
2 df.index      # DatetimeIndex(['2026-07-01', ...])
3
4 pd.date_range("2026-01-01", periods=5, freq="B")
5 # 从1月1日起的5个工作日 (B = Business day)
```

- `pd.to_datetime()`: 把字符串转成真正的日期类型。
- 变成日期后, 可以按年、月、星期智能取值与聚合。
- `date_range` 生成规则日期序列, `freq="B"` 表示周一至周五, 不排除交易所节假日。

日期索引的威力

`df.loc["2026-07"]`: 直接取出 2026 年 7 月的全部数据。

```
1 df.pct_change(fill_method=None)      # 每列的日收益率: (今日-昨日)/昨日
2 df.shift(1)                          # 整表向下错位一行 -> 昨日价格
3 df.diff()                            # 价格变动额 (今日-昨日)
```

- pct_change(): 把第 3 章的手写公式封装成了一个方法, 整张表所有列一次全算。
- shift(1): 错位是金融计算的核心技巧——“昨日收益”信号与次日收益配对’都靠它。
- 第一行没有昨日数据, 结果自动为 NaN (缺失)。

```
1 df.resample("ME").last()    # 每月最后一个交易日的价格
2 df.resample("ME").mean()    # 每月平均价格
```

- `resample(频率)`：把时间序列改换频率，像“汇总报表”。
- 常用频率：“D” 日、“W” 周、“ME” 月末、“YE” 年末。
- 后面必须接聚合方式：`last/mean/sum` 等。

典型应用

把日度净值换算成月度收益报告；把分钟线合成日线。

10. 缺失值处理



```
1 import numpy as np
2 s = pd.Series([1500.0, np.nan, 1515.0])
3
4 s.isna()          # [False, True, False] 找出缺失
5 s.dropna()        # 删掉缺失的行
6 s.fillna(s.mean()) # 用均值填充
7 s.ffill()         # 用前一个有效值填充（金融常用！）
```

- NaN (Not a Number) 表示缺失：停牌、数据未披露、pct_change 的第一行。
- ffill() (前向填充)：沿用上次观测值；是否适合需依据数据含义判断。
- 处理前先想：缺失代表什么经济含义？乱填会引入偏差。

11. 读写数据



```
1 df = pd.read_csv("../data/eod_data.csv",          # 文件路径
2                 index_col=0,                    # 第0列作为索引
3                 parse_dates=True)               # 把索引解析为日期
```

- 真实行情/财报数据多以 CSV、Excel 形式存在。
- 读入时顺手完成“第一列当日期索引”，后面全受益。
- 导出同样简单：df.to_csv("../outputs/result.csv")。
- Excel：pd.read_excel("../data/data.xlsx")（需安装 openpyxl）。

路径小贴士

从 notebooks/ 运行时，../ 表示本章目录；
输入放 data/，生成结果放 outputs/。

12. 综合案例与小结



```
1 rng = np.random.default_rng(42)
2 dates = pd.date_range("2026-01-05", periods=250, freq="B")
3
4 # 模拟三只股票的日对数收益率，再还原为价格
5 rets = pd.DataFrame({
6     "茅台": 0.0004 + 0.015 * rng.standard_normal(250),
7     "平安": 0.0002 + 0.020 * rng.standard_normal(250),
8     "招行": 0.0003 + 0.018 * rng.standard_normal(250)},
9     index=dates)
10 prices = 100 * np.exp(rets.cumsum())
11 prices.tail(3)          # 查看最后3天
```

- 用 DataFrame 字典一次性构造三列，索引是简化的工作日序列。
- cumsum、exp 对整张表向量化生效——NumPy 功力再现。

```
1 rets = prices.pct_change(fill_method=None).dropna()    # 日收益率
2
3 rets.describe().round(4)                                # 统计摘要
4 rets.mean() * 252                                       # 年化平均收益
5 rets.std() * np.sqrt(252)                               # 年化波动率
6 rets.corr().round(2)                                    # 相关系数矩阵
```

- 一个方法作用于**所有列**：这就是 DataFrame 的威力。
- **相关系数矩阵**：衡量资产间联动性，组合分散化的依据。
- 从构造数据到风险报告，没有写一个循环。

课堂讨论

相关系数接近 1 的两只股票，能互相分散风险吗？

- **两种结构**: Series (一系列带标签)、DataFrame (一张表); 行 = 日期, 列 = 资产。
- **选取**: `df[列名]` 取列; `loc` 按标签、`iloc` 按位置; 布尔条件筛行。
- **排序分组**: `sort_values/sort_index`; `groupby` 三步走。
- **时间序列**: `to_datetime`、`pct_change`、`shift`、`resample`。
- **缺失值**: `isna/dropna/fillna/ffill`。
- **读写**: `read_csv(index_col=0, parse_dates=True)`。
- **金融工具箱**: `describe()`、年化波动率 $\sigma \times \sqrt{252}$ 、相关矩阵 `corr()`。

课后任务

- ① 运行课堂 notebook, 替换 freq、种子等参数观察输出。
- ② 对案例数据：找出每只股票的最大单日跌幅及其日期（提示：idxmin() 返回最小值所在的索引）。
- ③ 选做：用 resample("ME").last() 得到月末价格，再算月度收益率。

下章预告

数字算出来了，怎么“看”得清楚？

下一章学习 Matplotlib **可视化**：价格曲线、收益分布、K 线之外的图表语言。

本章内容改编自教材第 5 章^[1]。

- Wes McKinney, *Python for Data Analysis*, 3rd ed.
- <https://wesmckinney.com/book/pandas-basics>
- 新增中文说明、合成数据与练习为本课程自行编写。
- 阅读正文理解概念；在配套 Notebook 中运行、修改、核对。



- [1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京: 人民邮电出版社, 2018.



Thanks for your attention!

Q & A

